



UNIVERSITÀ
DI TRENTO

HiCREST
HIGH PERFORMANCE COMPUTING AND RELIABLE SYSTEMS TRENTO

GPU Computing

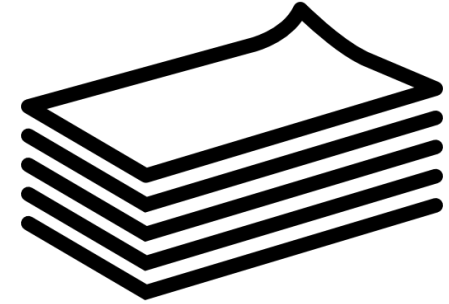
C lang recap and benchmark metrics

Prof. Flavio Vella and Lorenzo Pichetti
High-Performance Computing and Reliable System Laboratory

Stack memory Vs. Heap memory

We categorized RAM memory into two classes:

Stack memory: like a spot for a stack of papers. You can't interact with the lower-level sheets if you don't remove the one on the top.



Heap memory: like rooms in a hotel. When you check-in (**allocating memory**) you get some key copies, when you check-out (**freeing memory**), you give the keys back. While you own the room, all the guests with a key can access it. Once you checked-out, the memory won't be accessible anymore.



Stack and Heap example

Stack variables are declared by typing them into the function.

Heap variables are allocated and freed with “malloc” and “free”.

Note: every variable on the heap needs a pointer on the stack to be accessed.

```
int main(void) {  
    int input = 5;  
    fprintf(stdout, "Factorial on the stack: %d\n", stack_factorial(input));  
    int *result = (int*)malloc(sizeof(int));  
    heap_factorial(input, result);  
    fprintf(stdout, "Factorial on the heap: %d\n", *result);  
    free(result);  
    return 0;  
}
```

```
int stack_factorial(int n) {  
    int r = 0;  
    if (n > 1)  
        r = n * stack_factorial(n-1);  
    else  
        r = 1;  
    return(r);  
}
```

```
void heap_factorial (int n, int* r) {  
    if (n > 1)  
        heap_factorial(n-1, r);  
    else  
        *r = 1;  
    (*r) *= n;  
    return;  
}
```

Exercise 1a

Follow the directives in 'laboratories/Laboratory_II/ex1/stack_n_heap.c'. Solve the first four subproblems where you perform an elementwise sum between stack/heap vectors.

0	1	2
---	---	---

+

2	1	0
---	---	---

=

2	2	2
---	---	---

More detailed instructions are included in the README.md file

Matrix linearization

In C, **matrices are arrays** or arrays of pointers. We will always consider full linearized matrices (we will never use arrays of pointers).

My abstract matrix A

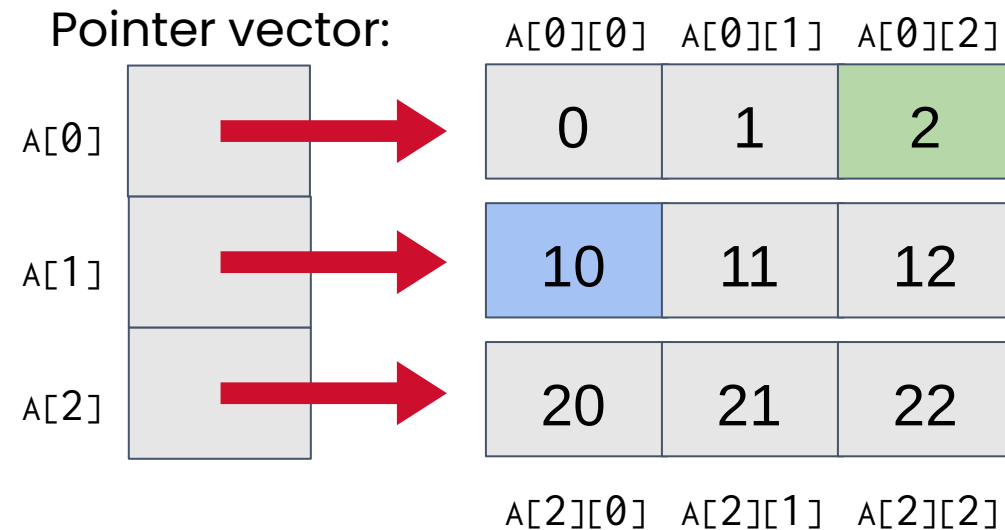
0	1	2
10	11	12
20	21	22

Note: in 2D representation, the different rows could be stored in memory addresses far apart.

1D representation

$A[0*m+0]$	$A[0*m+1]$	$A[0*m+2]$	$A[1*m+0]$	$A[1*m+1]$	$A[1*m+2]$	$A[2*m+0]$	$A[2*m+1]$	$A[2*m+2]$
0	1	2	10	11	12	20	21	22

2D representation



Exercise 1b

Complete the last four subproblems where you perform an elementwise sum between stack/heap matrices.

0	1	2
10	11	12
20	21	22

 $+$

1	1	1
1	1	1
1	1	1

 $=$

1	2	3
11	12	13
21	22	23

Benchmark metrics

How to get the runtime of a program

CPU timers

Gettimeofday is a C function included in “<sys/time.h>” that measures the number of seconds and microseconds elapsed since the Unix epoch (January 1, 1970).

The difference between two measurements gives us the elapsed time with **microsecond precision**.

```
#include <stdio.h>
#include <sys/time.h>

int main(void) {
    struct timeval temp_1={0,0}, temp_2={0,0};
    gettimeofday(&temp_1, (struct timezone*)0);
    [ ... some operation ... ]
    gettimeofday(&temp_2, (struct timezone*)0)
    double time = ((temp_2.tv_sec-temp_1.tv_sec)*1.e6+(temp_2.tv_usec-temp_1.tv_usec));
    fprintf(stdout, "Elapsed time: %lf\n", time);
    return 0;
}
```

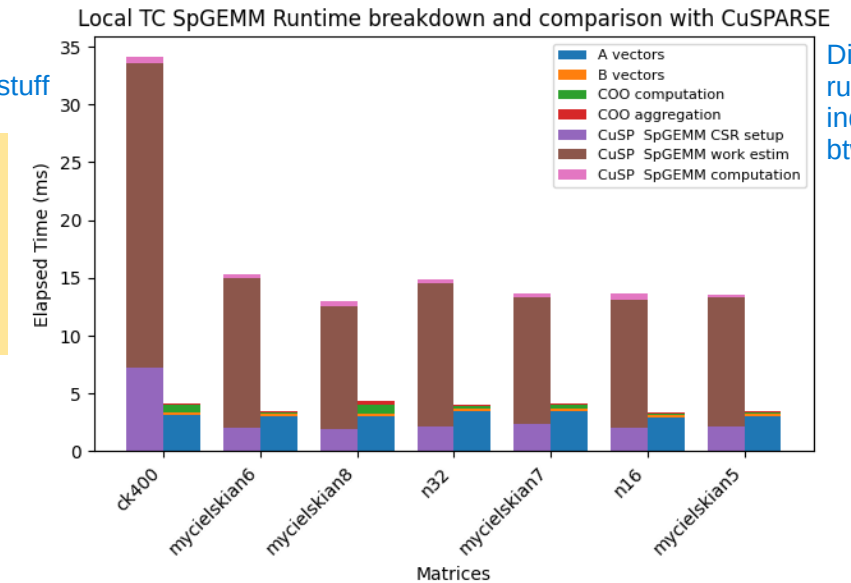
Reliable results

Ain't good that one --->
Because I'm loading drivers and doing other stuff

External phenomena (network noise, shared resources access...) can affect execution time. To mitigate them:

- **Mean of multiple runs:** Averaging execution times across multiple runs provides a more representative measure.
- **Standard deviation of each repetition:** Measuring the variation helps assess consistency and detect anomalies.
- **Warm-up cycles:** Discarding some runs before recording execution times helps stabilize performance.

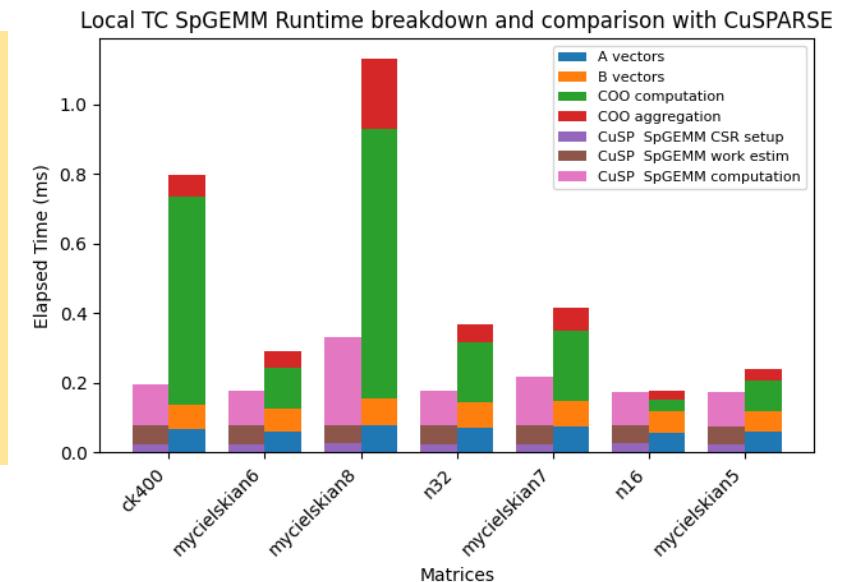
One-shot



Different runs are independent btw

So in the report, it's better if we do not only one try, but instead we go multiple times in time

Warm-up & average



Exercise 2

Given a command-line parameter n , generate two random vectors a and b of length 2^n and compute their sum $c = a + b$. Benchmark its runtime by performing two warm-up cycles and averaging over 10 rounds.

```
int main(void) {  
    [ ... ]  
    for(i = -WARM_UP; i<NITER; i++) {  
        [ ... timer start ... ]  
        c = add (a,b);  
        [ ... timer stop ... ]  
        if (i>0)  
            mesures[i] = [ ... time_stop - time_start ... ];  
    }  
    [ ... compute mean and std ... ]  
    return 0;  
}
```

Floating point operations

A common way to measure the performance of high-performance systems is through **FLOP/s** (Floating Point Operations Per Second). It quantifies the system's ability to perform mathematical calculations.

How to compute FLOP/s:

- 1. Count the floating-point operations:** number of floating-point operations (additions, multiplications, etc.) performed by the system during a specific task.
- 2. Measure the time:** Measure the time elapsed to complete the task.
- 3. Compute the ratio:**

$$FLOP/s = \frac{\text{Number of operations}}{\text{Time elapsed}}$$

Exercise 3

Given two random vectors of the same size 2^n , compute their scalar product:

$$\langle \vec{a}, \vec{b} \rangle = \sum_{k=1}^{2^n} a_i \cdot b_i$$

Get the execution time and FLOP/s. How much operations do a scalar product require?

Collecting results on different inputs

The **arithmetic mean** is used to average results on the same input instance. To aggregate runtimes across different input instances, the **geometric mean** is preferred, as it balances variations and prevents extreme values from skewing the results.

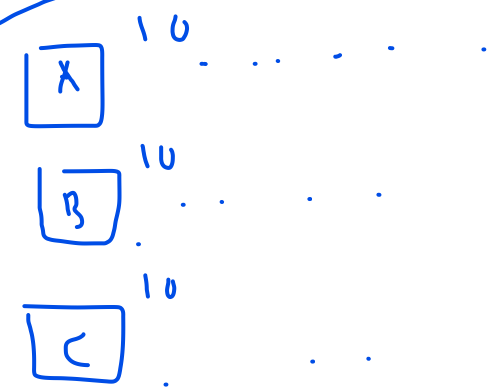
Given n measures $\vec{x} = (x_1, x_2, \dots, x_n)$:

Arithmetic mean:

$$AM(\vec{x}) = \frac{1}{n} \sum_{i=1}^n x_i$$

Geometric mean:

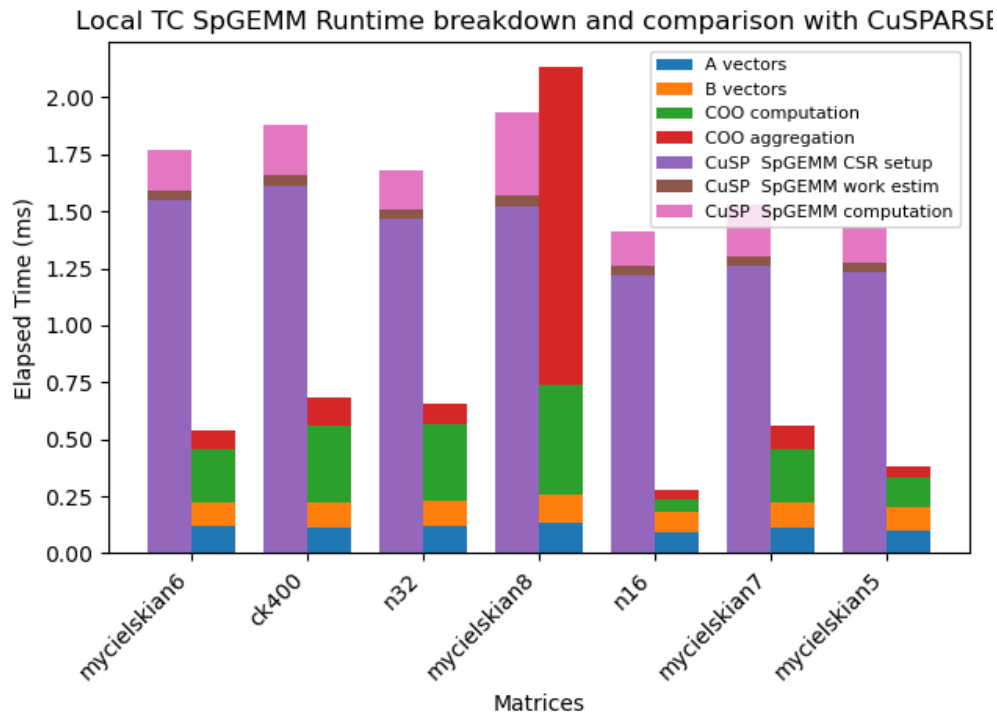
$$GM(\vec{x}) = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$



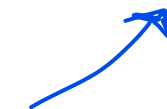
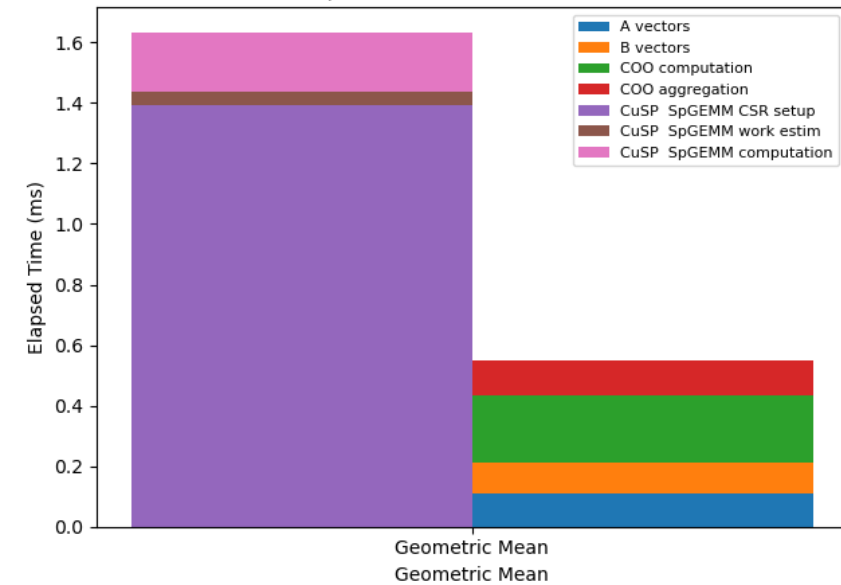
Exercise 4

Generate three vectors with different mean μ_1, μ_2, μ_3 and standard deviation $\sigma_1, \sigma_2, \sigma_3$.
Compute the overall average with both arithmetic and geometric mean.

COMMON USAGE EXAMPLE (this is not the exercise output):



Geometric Mean of Local TC SpGEMM Runtime breakdown and CuSPARSE com



It's better to do the GEOMETRIC MEAN <----- check why

Example: GEMM

Very important for the course

General Matrix Multiplication

General Matrix Multiplication (GEMM)

Given three integers n , k and m and two matrices $A \in M_{n,k}$ $B \in M_{k,m}$ (where $M_{i,j}$ is the set of matrices with i rows and j columns).

The **matrix product** $AB = C \in M_{n,m}$ is defined as:

$$C_{i,j} = \sum_{h=1}^k A_{i,h} \cdot B_{h,j} = \langle A_{i,\cdot}, B_{\cdot,j} \rangle$$

Each element (i,j) of the Matrix C is the scalar product between the i -th row of A and the j -th column of B .

Exercise 5a

Benchmark a program that use **OpenBLAS** to perform a general matrix multiplication between two random square matrices. **Get the execution time and the FLOP/s.**

5	0	0	2
1	0	2	0
0	0	4	0
-3	0	0	6

 $\times$

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

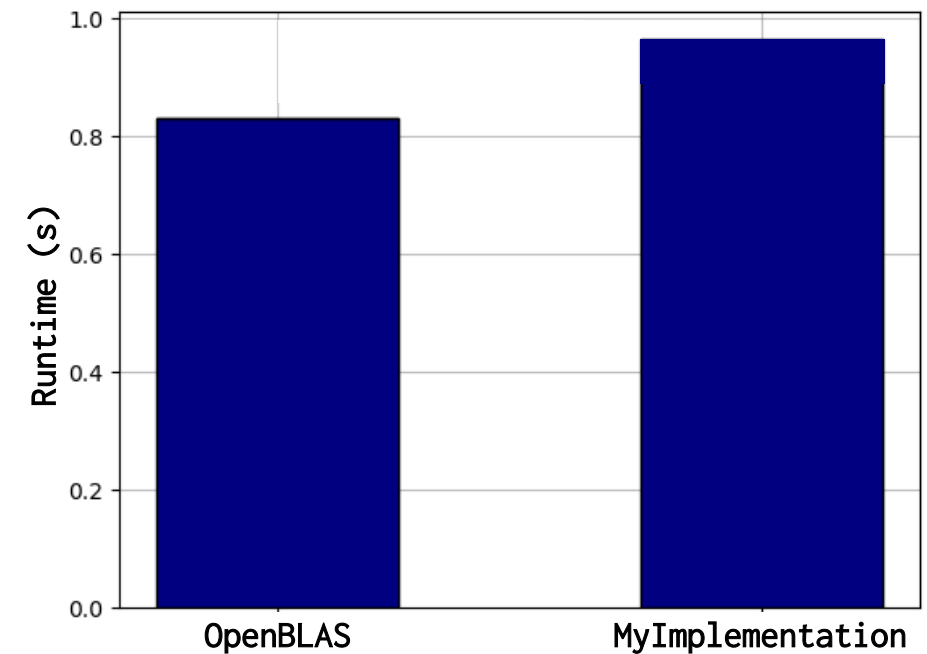
 $=$

7	7	7	7
3	3	3	3
4	4	4	4
3	3	3	3

Exercise 5b

Implement your own general matrix multiplication between two random square matrices and benchmark your result with respect to the OpenBLAS one.

Use OpenBLAS to perform a **correctness check!**

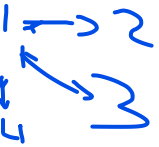


Example: COO sparse format

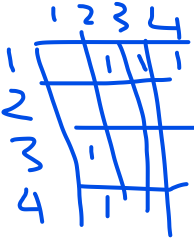
How to represent sparse matrices

The COO sparse matrix format

Very famous cuz it can represent graphs, biological problems



A sparse matrix is a matrix where the number of **nonzero elements** is much smaller than the **total number of elements** ($\text{nnz} \ll \text{rows} \times \text{cols}$).



To store them efficiently, several formats exist. The *coordinate* matrix format (**COO**) is easier one.

A

	0.5			1.2	
		-1.0			
2.3					0.2
			4.5		
	-3.0			6.1	

Given a sparse matrix A, we represent it with three vectors:

1. The first two array will store the coordinate of each nnz value:

Arow:

0	0	1	2	2	4	5	5
---	---	---	---	---	---	---	---

Acol:

1	4	2	0	5	3	1	4
---	---	---	---	---	---	---	---

2. The third array will store all the values:

Aval:

0.5	1.2	-1.0	2.3	0.2	4.5	-3.0	6.1
-----	-----	------	-----	-----	-----	------	-----

Exercise 6

Generate a dense $n \times m$ matrix where each entry has a probability p of being 0.

Write two functions:

1. Convert a dense matrix to COO format.
2. Convert a COO matrix to dense format.

Benchmark both functions for different values of p and aggregate the results using the **geometric mean**. What are the differences between function 1 and function 2?